BITS PILANI, DUBAI CAMPUS
SECOND SEMESTER 2025 – 2026
Department of Computer Science
Semester-Long Integrated Assignment

Course Code: CS F363 | **Max Marks: 60**
Course Title: Compiler Construction | **Weightage: 30%**
Mode of Assessment: Open Book | **Nature: Team-Based**

==========

Design and Implementation of a Mini Compiler

The objective of this assignment is to design and implement a working mini compiler covering all classical phases of compilation. The assignment is cumulative and must demonstrate a complete compilation pipeline from source program to target code.
The compiler must implement:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Symbol Table Management
5. Intermediate Code Generation
6. Basic Code Optimization
7. Target Code Generation

Each phase must build upon the previous phase.

This is a group assignment to be completed in teams of four. Each team is required to design and implement a mini compiler for the prescribed core language specification. While the course covers Lex and Yacc, teams are free to implement their compiler in any programming language of their choice (e.g., C, C++, Java, Python, etc.). The choice of implementation language will not affect evaluation, provided all required compiler phases are correctly implemented and integrated. The compiler must demonstrate a complete compilation pipeline and process the uniform evaluation program provided.

I. Instructions:

1. **Team Structure:** Each team can have Max of 4 members. All team members are expected to contribute.

2. **Evaluation:** This assignment contributes 30% to the overall course grade (60 marks total). It comprises Mid-Sem (20 marks, 10%) and Comprehensive (40 marks, 20%) evaluations. A total of 15 marks are allocated for viva, distributed as 5 marks (2.5%) during Mid-Sem and 10 marks (5%) during Comprehensive. The viva marks are awarded individually, while the remaining marks are based on technical evaluation of the group submission.

3. **Language Specification:** To ensure uniform evaluation, all teams must implement the following core language.

   a. **Data Types:**
      i. int
      ii. float
   b. **Mandatory Constructs: Every compiler must support:**
      i. Variable declaration

ii. Assignment statement
iii. Arithmetic expressions (+, -, *, /, %)
iv. Relational operators (<, >, <=, >=, ==, !=)
v. Boolean operators (&&, ||, !)
vi. if–else statement
vii. while loop
viii. Block structure using { }
ix. print statement

4. **Programming Construct for Evaluation*:** All teams must successfully compile the following program:

```
int a;
int b;
int sum;
float avg;

a = 2 * (3 + 4);
b = 15;
sum = 0;

while (a < b && b != 0) {

int temp;
temp = a * 2;

if ((temp % 3 == 0) || (a > 5)) {
sum = sum + temp;
} else {
sum = sum - 1;
}

a = a + 1;
}

avg = sum / (b - a);

if (!(avg < 5.0)) {
print(sum);
} else {
print(avg);
}
```

II. Midsem Evaluation (10% - 20 Marks):

Scope: Lexical Analysis and CFG-Based Syntax Analysis

Based on the prescribed core language specification and the mandatory evaluation program, each team is required to complete the following tasks.

Question 1: Lexical Specification and Tokenization (5 Marks)

Date of Evaluation: 02/03/2026 - 05/03/2026

Using the given language specification (int, float, control constructs, operators, etc.), formally define the lexical structure of the language.

You are required to:

- Identify and classify all token categories relevant to the prescribed constructs.

- Specify the regular expressions corresponding to each token class.
- Implement a lexical analyzer that reads a source program and generates a token stream.
- Demonstrate the token stream generated for the prescribed evaluation program.
- Clearly report lexical errors, if any.

Your lexical analyzer must correctly tokenize the entire evaluation program provided.

Question 2: Grammar Design and Syntactic Validation (7 Marks)
Date of Evaluation: 16/03/2026 - 20/03/2026

Construct a complete Context-Free Grammar (CFG) that generates the prescribed core language, including:

- Declarations
- Assignment statements
- Expressions
- Boolean expressions
- if–else statements
- while loops
- Block structures

The grammar must be capable of generating the entire evaluation program without modification.
You must then demonstrate syntactic validation by:

- Showing leftmost derivation for at least one non-trivial statement from the evaluation
- program.
- Showing rightmost derivation for at least one non-trivial statement.
- Constructing the corresponding parse tree.
- The syntax analyzer must consume the token stream produced by your lexical analyzer.

Question 3: Syntax Error Detection (3 Marks)
Date of Evaluation: 23/03/2026 - 27/03/2026

Demonstrate that your syntax analyzer can detect and report syntactic errors. Introduce at least two intentional syntax errors into the evaluation program and show the corresponding error messages produced by your system.

Mid-Sem Viva (5 Marks – Individual)
Date of Evaluation: 30/03/2026 – 02/04/2026

Each member will be evaluated individually on:

- Understanding of token definitions and regular expressions
- Grammar structure and precedence handling
- Ability to manually derive or trace parsing steps
- Explanation of their contribution to the implementation

III. Compre Evaluation (20% - 40 Marks):
Scope: Semantic Analysis to Target Code Generation

Using the same compiler developed for Mid-Sem, extend your implementation to complete the remaining compiler phases.

Question 1: Parser Implementation (10 Marks)
Date of Evaluation: 06/04/2026 – 10/04/2026

Upgrade your syntax analyzer by implementing two of the following:

- LL(1) parser
- Shift-Reduce parser
- SLR/LR(0) parser

You must provide:

- FIRST and FOLLOW sets
- Parsing table construction
- Parsing stack trace for a non-trivial input

The parser must correctly process the evaluation program.

Question 2: Symbol Table and Scope Handling (5 Marks)
Date of Evaluation: 13/04/2026 – 16/04/2026
Design and implement a symbol table capable of:

- Storing variable name, type, scope, and memory offset
- Supporting insertion and lookup
- Handling nested block scopes

Demonstrate symbol table updates while processing the evaluation program.

Question 3: Semantic Analysis (5 Marks)
Date of Evaluation: **13/04/2026 – 16/04/2026**
Using the parse tree or abstract syntax tree, implement semantic analysis for the prescribed language. Your system must detect and report:

- Use of undeclared variables
- Multiple declarations within the same scope
- Type mismatches in assignment and expressions
- Invalid boolean conditions

Demonstrate semantic validation for the evaluation program and show at least two semantic error cases.

Question 4: Intermediate Code Generation (5 Marks)
Date of Evaluation: 20/04/2026 - 23/04/2026
Generate Three-Address Code (TAC) for the entire evaluation program. Your implementation must correctly handle:

- Arithmetic expressions
- Boolean expressions
- Conditional branching
- Loop control flow

Temporary variables and jump labels must be clearly shown.

Question 5: Optimization and Target Code Generation (5 Marks)
Date of Evaluation: 20/04/2026 - 23/04/2026
Apply at least one basic optimization technique to the generated intermediate code. Then generate corresponding target code in the form of Pseudo assembly code. The generated target code must preserve program semantics.

Comprehensive Viva (10 Marks – Individual)
Date of Evaluation: 04/05/2026 - 07/05/2026
Each member must demonstrate:

- Symbol table trace
- Semantic validation reasoning
- Parsing stack simulation (if applicable)
- TAC generation for loop and conditional
- Control flow translation to target code

Marks will be awarded based on depth of understanding and ability to explain implementation.

***Note:**
Failure to correctly process the prescribed evaluation program will result in deduction in both Mid-Sem and Comprehensive evaluations.